

DESARROLLO DE APLICACIONES MÓVILES PROYECTO ANDROID

Aplicación de Chat Android con Firebase

1. Descripción general

El proyecto consiste en desarrollar una aplicación móvil nativa para Android que permita la comunicación entre usuarios mediante mensajería en tiempo real utilizando Firebase.

El proyecto deberá realizarse en grupos de máximo dos (2) estudiantes y será presentado de forma presencial en las fechas establecidas por el profesor. Será obligatorio utilizar GitHub como repositorio del código fuente.

Cada estudiante será evaluado de manera individual considerando su participación, los aportes realizados al repositorio, el conocimiento del código y su capacidad para explicar las decisiones tomadas durante el desarrollo.

Valor del proyecto: 20 puntos.

2. Tecnologías y lineamientos

- Java/Kotlin como lenguaje de programación.
- XML Views para la construcción de las interfaces.
- Activities y/o Fragments para la organización de las pantallas.
- RecyclerView para la presentación de listas y del historial de mensajes.
- Arquitectura MVVM para separar responsabilidades.
- Firebase Authentication para registro y autenticación de usuarios.
- Firebase Firestore o Realtime Database para la mensajería en tiempo real.
- Firebase Storage para el almacenamiento de imágenes.
- Firebase Cloud Messaging (FCM) para las notificaciones push.
- Git y GitHub para control de versiones y seguimiento de contribuciones.

3. Requisitos mínimos obligatorios

Los siguientes requisitos son independientes del porcentaje de funcionalidades completadas. Su incumplimiento implica que el proyecto no cumple con las condiciones mínimas establecidas.

1. Desarrollo nativo Android utilizando Java/Kotlin.
2. Interfaces desarrolladas utilizando XML Views.
3. Uso adecuado de Activities y/o Fragments e Intents.
4. Uso de RecyclerView para mostrar el historial de mensajes.
5. Implementación de arquitectura MVVM, manteniendo separación entre interfaz, lógica de presentación y acceso a datos.

6. Uso de Git y GitHub durante el proceso de desarrollo.
7. Ambos integrantes deberán presentar contribuciones verificables en el repositorio.
8. Cada integrante deberá demostrar conocimiento y comprensión del código durante la presentación.
9. Cumplimiento de al menos el 80% de los requerimientos funcionales puntuables.

4. Requerimientos funcionales y evaluación

4.1 Registro, autenticación y validaciones — 15%

- Registro de usuarios mediante correo electrónico y contraseña utilizando Firebase Authentication.
- Inicio y cierre de sesión.
- Mantener la sesión iniciada al cerrar y volver a abrir la aplicación, salvo que el usuario haya cerrado sesión.
- Validación de campos obligatorios, formato de correo y contraseña.
- Presentación de mensajes de error comprensibles ante fallos de autenticación o validación.

4.2 Mensajería en tiempo real — 30%

- Enviar y recibir mensajes en tiempo real.
- Almacenar los mensajes utilizando Firebase Firestore o Realtime Database.
- Mostrar el nombre del usuario y la fecha/hora de envío.
- Ordenar correctamente el historial de conversación.
- Evitar el envío de mensajes vacíos.

4.3 Interfaz, RecyclerView y navegación — 15%

- Construir las interfaces mediante XML Views.
- Utilizar RecyclerView para mostrar los mensajes.
- Implementar navegación adecuada entre las diferentes pantallas.
- Utilizar correctamente Activities y/o Fragments.
- Utilizar Intents cuando corresponda.
- Manejar adecuadamente los eventos de la interfaz.
- Como mínimo, la aplicación deberá contemplar el flujo: Login → Registro → Conversaciones/Usuarios → Chat.

4.4 Arquitectura MVVM — 15%

- Implementar una separación clara de responsabilidades utilizando MVVM.
- Como mínimo, separar View (Activity/Fragment), ViewModel y Repository/acceso a datos.
- Evitar concentrar la lógica de negocio y el acceso a Firebase directamente en Activities o Fragments.
- No se requiere implementar Clean Architecture.

4.5 Envío y recepción de imágenes — 10%

- Seleccionar una imagen desde el dispositivo.
- Enviar imágenes mediante el chat.
- Almacenar los archivos utilizando Firebase Storage.
- Visualizar correctamente las imágenes recibidas dentro de la conversación.

4.6 Notificaciones push con FCM — 5%

- Implementar Firebase Cloud Messaging (FCM).
- Mostrar una notificación cuando el usuario reciba un nuevo mensaje, según el comportamiento definido para la aplicación.

4.7 Calidad del código y Git/GitHub — 10%

- Organización adecuada del proyecto y de los paquetes.
- Nombres claros y consistentes para clases, métodos y variables.
- Separación adecuada de responsabilidades.
- Uso continuo de Git durante el desarrollo.
- Commits identificables y distribuidos durante el proceso, no únicamente al finalizar.
- Participación verificable de ambos integrantes.
- Evitar código innecesariamente duplicado.

5. Distribución de la evaluación

Criterio	Peso
Registro, autenticación y validaciones	15%
Mensajería en tiempo real	30%
Interfaz, RecyclerView y navegación	15%
Arquitectura MVVM	15%
Envío y recepción de imágenes	10%
Notificaciones push con FCM	5%
Calidad del código y Git/GitHub	10%
TOTAL	100%

6. Mejoras opcionales

Las siguientes funcionalidades son opcionales y no forman parte del 80% mínimo de requerimientos funcionales. Podrán ser consideradas como evidencia de iniciativa y profundización, pero no compensan el incumplimiento de un requisito mínimo obligatorio.

- Chats grupales.
- Estado de conexión de los usuarios (en línea/desconectado).
- Cifrado de mensajes.
- Indicador de mensaje leído/no leído.
- Indicador de usuario escribiendo.
- Foto de perfil.
- Tema claro/oscuro.
- Búsqueda de conversaciones.
- Eliminación de mensajes.
- Envío de otros tipos de archivos.

7. Entrega y presentación

- El código fuente deberá entregarse mediante un repositorio de GitHub público o privado compartido con el profesor.
- La aplicación deberá poder ejecutarse y demostrarse durante la presentación presencial.
- Ambos integrantes deberán participar en la presentación.

- El historial de Git será utilizado como una de las evidencias para valorar la participación individual.
- La evaluación podrá ser diferente para los integrantes de un mismo equipo según el dominio demostrado y sus aportes.

8. Evaluación individual durante la defensa

Durante la presentación, el profesor podrá solicitar individualmente a cualquier integrante que explique una sección del código, describa el flujo de una funcionalidad, justifique una decisión de implementación o realice una modificación sencilla. El objetivo es comprobar que cada integrante comprende el trabajo entregado y participó activamente en su desarrollo.

9. Condición de aprobación

Para que el proyecto pueda considerarse aprobado, deberán cumplirse todos los requisitos mínimos obligatorios y, adicionalmente, deberá alcanzarse al menos el 80% de los requerimientos funcionales puntuables. Las funcionalidades opcionales no sustituyen los requisitos mínimos obligatorios.